

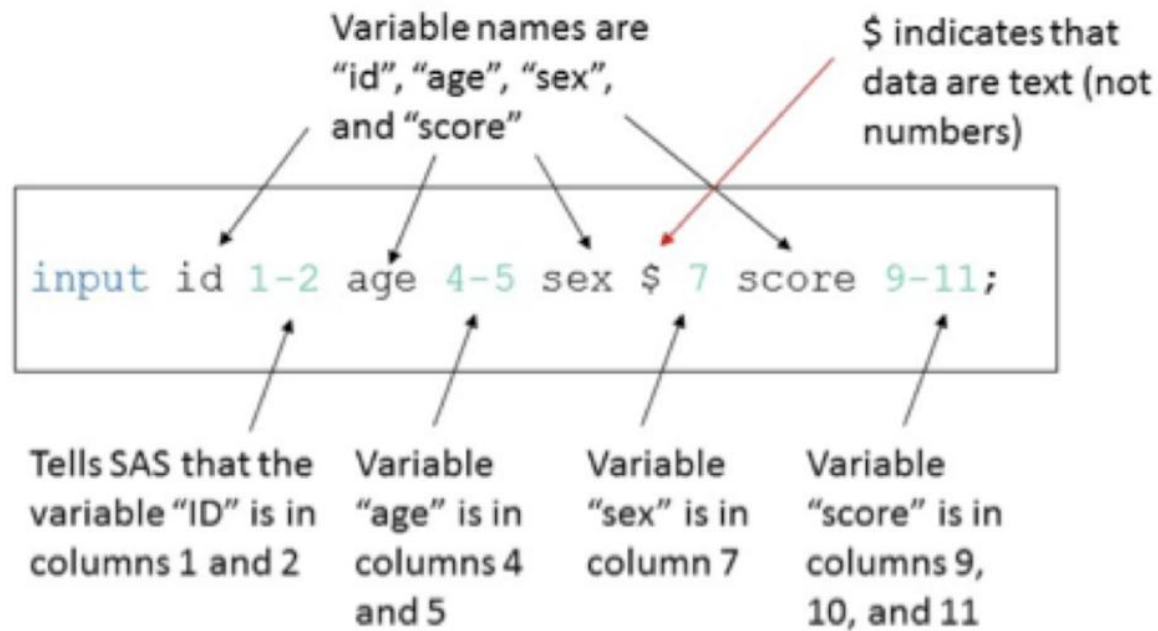
PHASE 1, MODULE 2: VARIABLES & MEMORY ARCHITECTURE

Khyber AI Systems Engineer (KASE)

Welcome back. Today we look under the hood of Python. To build AI systems that don't crash the server, you must understand how Python manages memory in your RAM.

Presented By: Zaryab Ahmad

THE 'LABEL' THEORY



Variables are Labels, Not Boxes

- In C++, a variable is a physical box in memory.
 - In Python, a variable is just a "name tag" tied to an object in memory.
 - Multiple name tags can be tied to the exact same object.
-

DYNAMIC TYPING

```
1 a = 2
2 b = 4
3 c = a + b
4
5 # La almohadilla '#' se usa para escribir comentarios, es decir
  cosas que no quieres que se ejecuten
6 # a, b y c son variables. Dentro del print() van separadas por
  comas
7 print(a, b, c)
8
9 # Lo que va entre comillas es texto
10 print(a, "+", b, "=", c)
```

<https://P01.jjdeharo.repl.run>

Python 3.6.1 (default, Dec 2015, 13:05:11)
[GCC 4.8.2] on linux
2 4 6
2 + 4 = 6
>

This Photo by Unknown Author is licensed under CC BY-SA

Python's Superpower (And Danger)

- You do not need to declare data types (e.g., `int x = 10`).
- Python figures out the type at runtime.
- A variable can change from a Number to a String instantly.

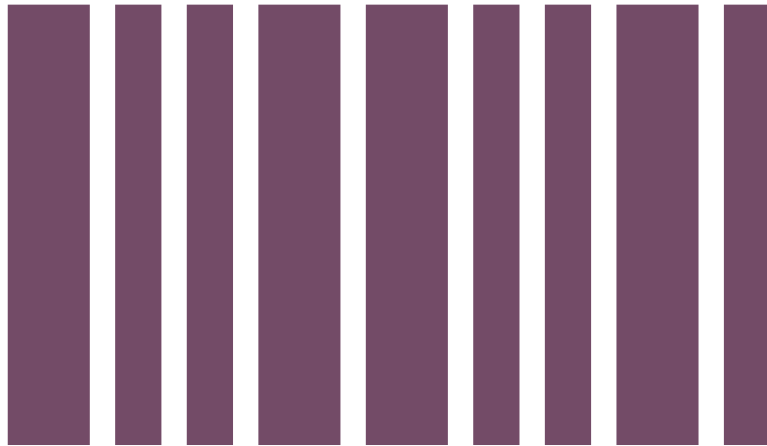
LOOKING AT RAM



The `id()` Function

- Every object created in Python gets a unique ID.
- This ID represents its physical address in your RAM.
- Use `id(variable_name)` to track where data lives.

THE JANITOR (GARBAGE COLLECTION)



Automated Memory Management

- Reference Counting: Python counts how many "labels" are attached to an object.
 - If the count drops to zero, the object is deleted to free up RAM.
 - This prevents "Memory Leaks" automatically.
-